

Fair Drop — Phase 0 Decisions & Assumptions

Status: **FINAL for Phase 1 start**. Changes need a dated entry in §5.

1. Repository assessment (2026-10-03)

- No Fair Drop code existed anywhere in the workspace. The scratch root held only unrelated projects (`achija-restaurant` , `anpr` , `signalpost_agent` , `tools` , `train_tracker`), and none were touched.
- New project root: `scratch/fair-drop/` . Phase 0 only adds `docs/` and a stub `README.md` .
- So there is nothing to keep or migrate. Every decision below is greenfield.

2. Final decisions

#	Decision	Chosen	Rejected alternative(s) and why
D1	Overall shape	Modular monolith : one Worker (API + static SPA) + one Durable Object class per drop	Microservices: too much operational surface for a hackathon, and no benefit at this scale
D2	Frontend hosting	Workers Static Assets (same Worker serves the SPA and /api/*)	Pages + separate Worker: cross-origin cookies/CORS for no gain
D3	Authoritative live drop state	DropDO SQLite storage (strongly consistent, transactional, single-threaded)	Writing D1 on every op: no transaction spanning DO+D1, higher latency, burns the D1 write quota
D4	Role of D1	System of record for cross-drop data (events, users, sessions, experiments, metrics, audit) and the final projection of each drop, written in batches through a DO outbox	D1 as the hot path
D5	DO topology	One DropDO per drop , <code>idFromName(drop_id)</code> . Each experiment run gets its own drop	Sharded registration DOs: deferred (see OD-3)
D6	Allocation algorithm	fairdrop-hmac-rank-v1 : commit-reveal server seed; final seed = H(server_seed snapshot_hash); rank = HMAC-SHA256(final_seed, participant_id); lowest N ranks win, the rest are an ordered waitlist	Fisher-Yates with a seeded PRNG: equally fair, but a single participant can't verify their own rank in isolation
D7	Ticket model	General admission, 500 ticket rows . Allocation grants an <i>offer</i> (entitlement). The ticket row is chosen when the hold is created	Pre-binding a ticket at allocation: more wasted state transitions on expiry
D8	Offer invariant	<code>active_offers + held + confirmed ≤ total</code> is enforced, so a valid offer can always be reserved	Overbooking offers: needs a compensation flow
D9	Identity	Lightweight account (display name + email, normalized and hashed for dedupe) + device ID + signed session cookie. No email verification in MVP	OAuth/OTP: adds infrastructure (see OD-1)
D10	Sessions	HttpOnly signed cookie (HMAC, <code>sid</code> , <code>uid</code> , <code>exp</code>), verified statelessly in the Worker; session row in D1 for recovery and revocation	Pure JWT with no server record: can't revoke or count concurrent sessions

#	Decision	Chosen	Rejected alternative(s) and why
D11	Rate limiting	Two tiers: edge = Workers Rate Limiting binding (per IP/network, approximate, per-colo); authoritative = token buckets in DropDO memory per session, account and participant	WAF rules only: can't key on session/account
D12	Idempotency	Semantic idempotency first (natural key = drop + participant + action). Idempotency-Key header is a secondary guard on mutating calls	Header-only idempotency: misses retries that use a new key
D13	Behavioral detection	Browser-side aggregation into ≤ 1 summary per 10 s, server-side network/session features, additive explainable score with per-category cap so no single signal can reach CHALLENGE	ML classifier: not explainable, no training data
D14	Challenge	Cloudflare Turnstile only as escalation (risk $\geq$ CHALLENGE). Simulated drops use a declared challenge oracle (pass probability per profile)	Turnstile on every booking: makes Turnstile the primary fairness mechanism, which is explicitly unwanted
D15	Live updates	WebSocket Hibernation API on DropDO . Aggregated broadcast every 500 ms plus a rate-capped sampled event feed	Polling D1: quota + latency
D16	Consumer status	Polling GET /drops/:id/me , with a server-dictated next_poll_ms + jitter. Public drop status is edge-cached for 1 s	Consumer WebSockets: unnecessary for consumers, costs DO requests
D17	Simulator	Python asyncio (httpx + HTTP/2). Logical mode (batched synthetic ops, virtual clock) and Real HTTP mode (real concurrent requests, wall clock, capped volume)	Browser automation at 50k: infeasible
D18	Sim clock	Logical drops use a simulator-driven virtual clock (monotonic virtual_now per batch). Real/consumer drops use wall clock + DO alarms	Real-time timers in logical mode: a 50k run would take real minutes and mix two clocks
D19	Label blindness	The server never receives HUMAN/BOT labels during a run. The simulator commits population_hash at creation and uploads labels only after allocation is final	Labels inline: would let detection cheat
D20	Baseline	naive-fcfs-v1 runs in the same DO with fairness controls switched off. Both modes share the same declared capacity/load-shedding model	A separate naive codebase: comparison not apples-to-apples
D21	Sim auth	Admin passphrase → short-lived admin cookie (UI). Experiment-scoped HMAC sim token for	Shared static API key everywhere

#	Decision	Chosen	Rejected alternative(s) and why
		simulator calls. Virtual client network headers are honoured only with a valid sim token on an experiment drop	
D22	Storage of 50k logical participants	Per-participant state in DO memory; persisted as chunk rows ($\leq 1,000$ participants/row) to stay inside free-tier row-write quotas. Consumer participants get individual rows	50k individual rows per run: ~50% of the daily DO write quota per run
D23	Repo layout	pnpm workspace: apps/worker , apps/web , packages/shared (types, zod schemas, allocation + verification code shared by server and browser), simulator/ (Python, uv)	—

3. Platform budget assumptions (Cloudflare Workers Free — verify in Phase 1)

Resource	Free limit (as researched 2026-10)	Design response
Worker requests	100k/day	Logical mode batches 500–1,000 ops/request. Real HTTP mode capped (default $\leq 20k$ requests/run)
Worker CPU	10 ms/request	The Worker only does auth, edge limits and routing. Heavy work runs in the DO
DO requests	100k/day (incl. WS incoming messages, alarms)	Same batching. Dashboards receive and don't send
DO CPU	30 s/invoke default	Allocation of 50k $\approx$ 50k HMACs, expected well under the limit. Chunkable via alarms if not
DO SQLite rows written	100k/day (index updates count)	Chunked logical participants, minimal indexes, counters in memory with checkpointing
D1 rows written	100k/day	Target $\leq \sim 2.5k$ D1 rows per 50k experiment run
D1 rows read	5M/day	Comfortable
Storage	5 GB	Comfortable

If the demo needs more than ~ 10 full 50k runs/day, upgrade to **Workers Paid (\$5/mo)**. No redesign needed.

4. Assumptions

- A1. One event (FUTUREFEST 2026), 500 GA tickets, max 1 ticket per participant.
- A2. Payment is mocked: `confirm` = payment success.
- A3. Demo timings (consumer drop): registration 15 min, offer window 10 min, hold 5 min. Experiment drops use configurable timings on the virtual clock.
- A4. Accounts can be created freely (no verification), so **Sybil resistance is partial**. This is stated in the threat model.
- A5. Logical-mode results are only as valid as the synthetic behavior model. The *allocation* result is a real server computation; *detection* results on synthetic telemetry are labelled "model-dependent".
- A6. Turnstile can't be solved by the simulator. Challenge outcomes in sim drops come from a declared oracle and are reported as assumptions, not measurements.
- A7. Every number shown in results comes from a real run. No placeholder or fabricated metrics are ever rendered (the UI shows "—" when there's no data).

5. Open decisions (revisit at the noted phase)

ID	Question	Default until decided	Revisit
OD-1	Add email OTP / OAuth for stronger identity?	No verification. Sybil residual documented	Phase 8 / 15
OD-2	Store synthetic sim accounts in D1 users ?	No, they live in DO chunks + artifacts only	Phase 11
OD-3	Shard registration intake across multiple DOs?	Single DropDO . Shard only if the Phase 11 real-HTTP run shows DO saturation	Phase 11 / 15
OD-4	Mix a public randomness beacon (drand round after freeze) into <code>final_seed</code> ?	Commit-reveal only. The grinding residual is documented	Phase 15
OD-5	Exact risk thresholds/weights	Values in architecture §5.3, to be tuned against human FP rate	Phase 7 / 15
OD-6	Store raw email for display?	Store only <code>email_hash</code> + optional masked display string	Phase 8
OD-7	Free vs Paid Workers plan for the demo	Free, with budget tracking. Upgrade if > ~10 full runs/day are needed	Phase 16
OD-8	NAIVE capacity-model default values	<code>per_tick_ms=100</code> , <code>capacity_per_tick=50</code> , plus one alternate setting reported	Phase 14

6. Change log

- 2026-10-03 — Initial Phase 0 decisions.